

Taylor & Francis
Taylor & Francis Group

Applied Artificial Intelligence
An International Journal

ISSN: 0883-9514 (Print) 1087-6545 (Online) Journal homepage: www.tandfonline.com/journals/uaai20

System Logs Anomaly Detection. Are we on the right path?

Ramona-Georgiana Albert

To cite this article: Ramona-Georgiana Albert (2025) System Logs Anomaly Detection. Are we on the right path?, Applied Artificial Intelligence, 39:1, 2440692, DOI: 10.1080/08839514.2024.2440692

To link to this article: https://doi.org/10.1080/08839514.2024.2440692

© 2024 The Author(s). Published with license by Taylor & Francis Group, LLC.

View supplementary material

Published online: 24 Dec 2024.

Submit your article to this journal

Article views: 4826

View related articles

View Crossmark data

Citing articles: 3 View citing articles

Full Terms & Conditions of access and use can be found at
https://www.tandfonline.com/action/journalInformation?journalCode=uaai20

APPLIED ARTIFICIAL INTELLIGENCE
2025, VOL. 39, NO. 1, e2440692 (31 pages)
https://doi.org/10.1080/08839514.2024.2440692

Taylor & Francis
Taylor & Francis Group

OPEN ACCESS Check for updates

System Logs Anomaly Detection. Are we on the right path?

Ramona-Georgiana Albert

Faculty of Computer Science, Alexandru-Ioan Cuza University, Iasi, Romania

ABSTRACT

System logs are universally used for monitoring user access, performance, and behavior in software applications. Large-scale industrial systems generate an immense volume of logs, which are difficult to handle with human capabilities. Therefore, an automated method is essential for filtering vast amounts of data. System log anomaly detection is crucial in the security field for identifying system failures, sophisticated internal attacks, and other deviations from the norm. This research area requires further development, as most Deep Learning solutions in the literature are semi-supervised. This poses a significant limitation since these solutions are impractical for large-scale ecosystems due to the high cost of labeling data. This paper introduces a method that replaces the supervised phase of semi-supervised methods with fully unsupervised heuristics, utilizing the elbow method, interquartile range, and Simulated Annealing. The unsupervised results are comparable to the semi-supervised State of the Art while demonstrating greater applicability in real-world applications. This work proposes a more suitable benchmark for the log anomaly outlier detection problem, where the training data include both normal and abnormal sequences and precede the test sessions in time. Additionally, it presents metrics on distinct log sequences to mitigate the impact of unbalanced anomaly types.

ARTICLE HISTORY
Received 14 September 2024
Revised 9 November 2024
Accepted 28 November 2024

Introduction

In today's world, in the Information Age, many aspects that surround us are constantly expanding to unimaginable scales. Information has become one of the most important parts of our lives, and computational systems are continuously evolving to keep up with the demands. Therefore, companies like Amazon, Google, or Microsoft provide large-scale distributed systems that require system engineers to supervise them constantly. As the massive amount of data and services managed by big tech companies is intractable using human capabilities, an automatic way should be employed to filter out vast quantities of data and turn it into information understandable and actionable by humans. The goals can be either to understand the usual behavior of

CONTACT Ramona-Georgiana Albert albert.ramona99@gmail.com Faculty of Computer Science, Alexandru-Ioan Cuza University, General Henri Mathias Berthelot 16, Iasi 700483, Romania

© 2024 The Author(s). Published with license by Taylor & Francis Group, LLC.
This is an Open Access article distributed under the terms of the Creative Commons Attribution License (http://creativecommons.org/licenses/by/4.0/), which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited. The terms on which this article has been published allow the posting of the Accepted Manuscript in a repository by the author(s) or with their consent.

a subject of interest and predict behavior in fields such as marketing, e-commerce, and finances, or the opposite problem, trying to detect deviations from the norm, generically named anomaly detection, having applications mainly in the security field.

The primary resource used in the monitoring process is represented by system logs. A log system aims to record significant events that could help to find the root cause of multiple types of incidents. The most frequent one is represented by system failures which need to be quickly fixed to respect service-level agreements and maintain customer trust. Another use case for investigating the logs is represented by the abnormal behavior caused by software bugs. This type of incident is also encountered in software projects where the testing phase is neglected in favor of redistributing the workload to improve other modules. Last, but not least, the system logs help the investigators to gather evidence in the process of mitigating a security attack.

Large industrial systems generate a massive amount of system logs that cannot be manually monitored. Currently, anomaly detection is a critical tool, which brings into the attention of supervised experts just the important incidents. Usually, the automated system informs the on-call engineers via alarms in a reactive manner that something has happened and needs remediation.

Context

In the early ages of computer systems, operators would manually inspect system logs using keywords or regular expressions. With the development of large industrial systems, manual monitoring was replaced with automated systems for detecting system anomalies. Classic machine learning algorithms were successfully developed and applied to the system log anomaly detection task.

Classic machine learning methods cannot scale to the amount of data needed for the system log anomaly detection task, resulting in a fast migration to deep learning alternatives. He et al. (2016) experiments with six classical machine learning algorithms for log-based anomaly detection and outlines common limitations shared by these methods, namely weak adaptability, inefficiency, and inflexible features. LogRobust (Zhang et al. 2019) also outlines the performance brought by deep learning methods compared to classical machine learning approaches. Therefore, many researchers resorted to neural network architectures to address the system logs anomaly detection problems.

A deep learning approach, DeepLog (Du et al. 2017), reports a very high detection accuracy for system log anomalies. However, starting from DeepLog, several hidden assumptions have been propagated through related work in this area, impeding the industrial application of anomaly detection algorithms. The most relevant concern is that the

anomaly detection model requires a supervised hyperparameter tuning phase to achieve good results. Moreover, DeepLog is trained using only system logs generated on the normal execution path, which means that no anomalies are included in the training dataset. As a consequence, labeling the training data is required for using this solution. These are all critical factors that heavily influence the performance metrics and results.

In real-world large-scale anomaly detection problems, labeled data come with a high cost and the labeling process would need to be done in the long term since the user behavior and the definition of "normal" are under continuous change. Therefore, anomaly detection models that require supervised fine-tuning and labels are not a suitable approach for industrial use.

Contribution

At the beginning of a new anomaly detection task, there are no labeled data, and an unsupervised approach is needed. The system logs at a big-tech company scale constitute massive amounts of data and labeling them comes at a very high cost. Moreover, unknown events may occur over time and the detection model needs to be able to also catch new anomalous behavior.

In this paper, we propose removing the supervised phase of semi-supervised methods and replacing it with fully unsupervised heuristics. This removes the need for getting labeled data and may also eliminate the human error inherent in labeling (Farzad and Gulliver 2020). A framework is designed based on semi-supervised State of the Art (SOTA) methods in the literature that use autoregressive language models. The supervised phase is replaced with three unsupervised heuristics to decide whether a log sequence is an anomaly. Furthermore, we report additional metrics independent of the volume of data, providing clearer indicators about the performance of anomaly detection models.

Related Work

Unsupervised Methods

Several of the classical unsupervised machine learning algorithms used for standard anomaly detection are applied to features extracted from log sequences and do not take into consideration the order of events.

PCA was used by Xu et al. (2009) to transform the log sequences in a vectorized form following an event count vector inspired by the bag-of-words approach. The first k principal components represent the normal space, and the remaining (n – k) components represent the anomaly space. For each new instance, the projection of the vectorized form is

computed on both subspaces, and the new instance is treated as an anomaly based on the distance to the anomaly subspace. Even though this method achieves important results, it does not take ordering into consideration.

Invariant Mining is also used in the system logs anomaly detection problem (Lou et al. 2010). The algorithm is based on checking whether the invariants generated at a previous step are satisfied for a new sequence, otherwise the sequence is considered an anomaly. Unfortunately, this method suffers from the same shortcomings as bag-of-words modeling because it does not consider the order of events.

CloudSeer (Yu et al. 2016) builds an automaton based on the normal execution flows and for prediction, it checks the logs against a set of automata. Unfortunately, it requires knowing apriori which log sequences are normal. Moreover, while the method reports impressive results for OpenStack logs (Du et al. 2017), this method does not work for other datasets such as Hadoop Distributed File System (HDFS) PCA where the patterns are more irregular.

K-means. The most representative work that uses k-means for system log anomaly detection (Muniyandi, Ramachandran, and Rajaram 2012) applies clustering and the C4.5 decision tree methods for the classification of normal and anomalous activities. The k-means algorithm is used to partition the training data into k different clusters using the Euclidean distance, each cluster representing a density region for normal or abnormal sequences. The supervised phase is used to refine the decision boundaries.

Local Outlier Factor (LOF) is an effective unsupervised outlier detection algorithm based on the density deviation of each datapoint from its neighbors. The LOF algorithm computes a relative density value measuring the local deviation of data. LOF was successfully used for detecting anomalies in traffic data (Ma, Ngan, and Liu 2016), exceeding the distance-based model, kNN.

Autoencoder. Farzad and Gulliver (2020) use two autoencoders in combination with an Isolation Forest (IF) model (Liu, Ting, and Zhou 2008) for unsupervised log message anomaly detection. The autoencoders are designed to obtain a compact representation of the input data for constructing the IF model that isolates the abnormal sequences. The experimental settings are closer to real-life conditions compared to the other methods that are taking advantage of labeled data.

Semi-Supervised Methods

Autoregressive language models forecast future events using the context of past events. These models are leveraged on the idea that output variables are linear dependent on priori values. The model estimates the probability distribution for the next token x_i by applying the conditional probability

$p(x_i|x_1, x_2, \ldots x_{i-1})$. The probability of a sequence is computed with the chain rule as presented below:

$$p(x) = p(x_1, x_2, \ldots, x_n) = \prod_{i=1}^{n} p(x_i|x_1, x_2, \ldots x_{i-1}) \quad (1)$$

Based on the probability distribution of the tokens predicted by the autoregressive model given a sequence from the testing dataset, we can rank the predictions and extract their position. If at least one ground truth token is not within the top-g predictions, the sequence is considered anomalous. The intuition behind this procedure is that normal data have tokens that are easier to predict, therefore having a higher probability, so their position will be in the top-g. On the other hand, anomalies have tokens that are harder to predict and therefore their position will be lower.

The following autoregressive architectures have a supervised phase to fine-tune the g parameter. A common mistake that affects the integrity of the reported results is represented by the fact that the g parameter is learned by optimizing the F1 score directly on the testing dataset and the same testing dataset is used for reporting the model results.

DeepLog (Du et al. 2017) is a popular solution in the deep learning area for system logs anomaly detection. The base of the architecture is represented by Long Short-Term Memory (LSTM), known to maintain the context during training (Hochreiter and Schmidhuber 1997). The architecture predicts the categorical distribution of the next event over the vocabulary. During inference, the model is used to predict the next event given a sequence of tokens. If the ground truth token has a low probability, it means that the event could not be learned during the training phase, indicating that the sequence might be anomalous. DeepLog reports good results on HDFS, BGL (Blue Gene/L) (Oliner and Stearley 2007), and OpenStack exceeding classical unsupervised methods.

LogAnomaly (Meng et al. 2019) is similar to the DeepLog procedure with the particularity that instead of log tokens, it uses numerical vector representations of log templates. The main module that sustains the LogAnomaly research work is inspired by Word2Vec (Mikolov et al. 2013) and uses the distributional lexical-contrast embeddings model (Nguyen, Schulte Im Walde, and Vu 2016) for computing word representations.

LogBERT (Guo, Yuan, and Wu 2021) proposes a semi-supervised framework for log anomaly detection based on Bidirectional Encoder Representation from Transformers. The anomaly verdict is based on the number of masks which are correctly predicted. This approach has the events from future timestamps available and takes advantage of bidirectional learning. LogBERT requires a supervised hyper-tuning phase to

achieve the best results. There is a lack of integrity in the experiments because the hyperparameters are tuned by optimizing the F1 score on the same testing dataset.

LogFiT (Almodovar et al. 2024) uses a pretrained BERT-based language model fine-tuned to recognize patterns occurring in normal logs. Despite having good results, it requires training on normal log data, limiting its applicability to unlabeled datasets.

MetaLog (Zhang et al. 2024) is a cross-system anomaly detection method based on globally consistent semantic embeddings. Capitalizing on the meta-learning paradigm, they report similar results with supervised anomaly detection models trained with 100% labeled data while using only 1% labeled logs.

LogELECTRA (Yamanaka et al. 2024) is a new semi-supervised log anomaly detection model that analyzes the semantics of log messages using natural language processing. It claims to achieve better generalization on unseen log messages that do not follow the same log template as the training data.

Problem Formulation

Anomaly or outlier detection is an important data mining application. The anomalies are datapoints drastically different from the rest of the constituents of the dataset (Samariya and Thakkar 2023). Outlier and anomaly are interchangeable terms and widely used in most related works.

The System logs represent the primary resource used in the monitoring process. A log system aims to record significant events that could help to find the root cause of multiple types of incidents.

A common formulation of the system logs anomaly detection problem can be found in previous works by combining the two previously defined terms: anomaly detection and log system (Du et al. 2017; Guo, Yuan, and Wu 2021; Meng et al. 2019; Nedelkoski et al. 2020; Yang et al. 2021; Zhang et al. 2019).

Let $x_i^{(j)} \in \mathcal{T}$ be a token.

- $x^{(j)} = (x_1^{(j)}, x_2^{(j)}, \ldots x_n^{(j)}) \in \mathcal{T}^n$ is an ordered sequence of symbols;
- $D = \{x^{(j)} | j = \{m\}\}$ is a set of sequences.

Input: D set of event sequences

Output: subset $A \subseteq D$ which contains abnormal event sequences

In the context of autoregressive language models, we can approximate the probability of the entire sequence by estimating the probability distribution for the next token x_i using the conditional probability $p(x_i|x_1, x_2, \ldots x_{i-1})$, taking advantage of the chain rule as follows:

$$p(x) = p(x_1, x_2, \ldots, x_n) = \prod_{i=1}^{n} p(x_i | x_1, x_2, \ldots x_{i-1}) \quad (2)$$

Anomaly Detection Heuristics

As mentioned in the previous section about autoregressive language models, the output of the architectures is a distribution of probabilities on the vocabulary represented by the set of event templates. We developed three heuristics to decide if a sequence is an anomaly or not based on the autoregressive model prediction.

The anomaly prediction logic adopted by many research papers from the system logs anomaly detection area (Du et al. 2017; Guo, Yuan, and Wu 2021; Zhang et al. 2019) is that the sequences that have at least an event that is not among top-g predictions of the model are considered anomalies. The intuition behind this procedure is that tokens which are easier to predict, will have a higher probability, so their position will be at the top, while anomalies have tokens that are harder to predict and therefore their position will be lower. The shortcoming of this parameter g is that works in literature select it in a supervised manner, sometimes even based on the f1 score from the testing dataset.

Making the autoregressive models applicable to unlabeled datasets in the industry can be done by replacing the supervised selection of the g parameter with a heuristic that selects a good g in an unsupervised manner.

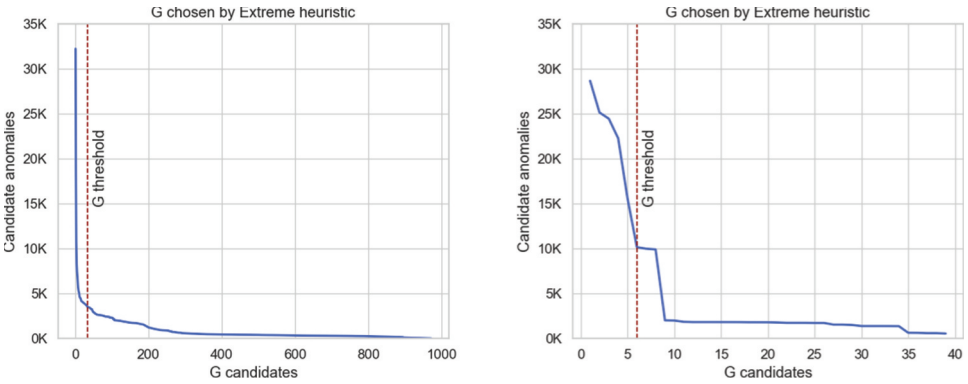
Extreme heuristic

The Extreme heuristic is a method of selecting the hyperparameter g based on the Elbow method. The idea behind the Elbow method is to monitor the number of anomalies predicted for each possible g (from 1 to the length of the dictionary $|\mathcal{T}|$). We display the results in Figure 1.

The Extreme heuristic automatically chooses the g hyperparameter that corresponds to the biggest drop in the number of predicted anomalies.

The advantage of the Extreme heuristic is clear: while the value may not be optimal, it has the advantage of choosing in an unsupervised manner, applying to a more general setting.

The Extreme heuristic also has shortcomings: there are multiple threshold candidates for g and the Elbow method is not suitable for all datasets. Notice how in Figure 1, for the HDFS dataset, there are two possible elbows: at $g = 6$ and $g = 9$. On the testing dataset a better F1 score is achieved for the second threshold at $g = 9$, however, our heuristic selected $g = 6$. This is why we came up with the next idea



(a) Elbow on BGL (∼1000 tokens). (b) Elbow on HDFS (19 tokens).

Figure 1. Anomalies curves for different values of hyperparameter *g*.

for an anomaly prediction heuristic entitled Global heuristic, which adds an additional layer to the decision-making process.

Global heuristic

The Global heuristic is meant to address the shortcomings of the Extreme heuristic, such as:

- There is only one point of failure; *g* is an extreme point attribute, meaning that if at least a token is predicted badly, the entire sequence will be considered anomalous;
- The False Positive (FP) rate is very big depending on the choice of the hyperparameter *g* the anomaly prediction should be more sensitive (based on multiple indicators).

Some sequences contain noise, but their overall behavior is a normal one. Such sequences would be incorrectly considered to be anomalies using the Extreme heuristic. To provide a solution to this problem, Global heuristic has been developed, which decides if a sequence is an anomaly or not based on the overall loss obtained by the autoregressive model.

DeepLog's architecture (Du et al. 2017) takes one token at a time and predicts the probability distribution for the next token. The LSTM cells maintain the context formed during training with previously seen tokens. The past information is retained in the weights of the neural network and helps the model to better predict the next tokens. We will treat the DeepLog architecture as a black box as it does not enter the scope of this paper.

Input

For LSTM-based models, the input for the model is a single token because the context is already stored in the model's weights. For transformer architectures, the entire sequence is given to the model to predict each token's probability distribution using the masking procedure.

Output and cross-entropy loss

The output is represented by the conditional probability of the next log key (LSTM-based models) or the masked tokens (transformer architectures). The most effective and popular loss used for probability distributions is the cross-entropy loss.

The cross-entropy loss penalizes the model's output represented by a probability distribution, based on how far it is from the actual expected probability distribution. In a classification problem, the target probability distribution is represented by a one-hot encoded vector, where one is in the position of the target in the vocabulary and zeros are for the rest of the tokens (e.g. $[0, 0, 0, 1, 0, 0]$ is the target for token 4 where $\mathcal{T} = \{1, 2, 3, 4, \langle \text{START} \rangle, \langle \text{END} \rangle\}$ is the dictionary).

$$L_{CE} = -\sum_{i=1}^{|\mathcal{T}|} t_i \cdot \log p(x_i) \tag{3}$$

- If the distance between the output of the model $p(x_i)$ and the targeted one-hot vector is large, the cross-entropy loss will be higher;
- Otherwise, the model prediction is close to the ground truth. This means that the model has already learned the instance and does not require a huge change in its parameters.

Global heuristic workflow

The loss for each token can be computed using the trained autoregressive model, and applying the interquartile range (IQR) method can extract anomalies diverging from the average of the cross-entropy losses for each test sequence. The IQR method is based on boxplots designed to provide a better understanding of a distribution. It offers information about the data's spread, maximum and minimum values, and the Q_1 and Q_3 quartiles (see Figure 2). Using this information, a threshold is selected for identifying which datapoints from the distribution are anomalous.

UW is the Upper Whisker, which represents the upper threshold for normal data.

$$\begin{aligned} \text{IQR} &= \text{Q3} - \text{Q1} \\ \text{UW} &= \text{Q3} + 1.5 \cdot \text{IQR} \end{aligned} \tag{4}$$

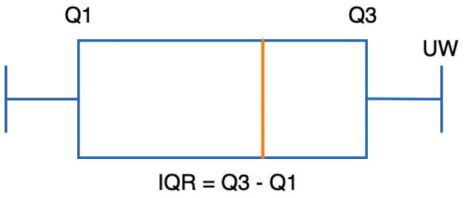


Figure 2. Example of Boxplot.

For a sequence $x^{(j)} = (x_1^{(j)}, x_2^{(j)}, \ldots, x_{k^{(j)}}^{(j)})$ of length $k^{(j)}$, the loss is computed as the average of the individual losses for each token in the sequence, excluding the first one. This is because the loss for each token x_l is calculated based on the prefix sequence up to that token $(x_1^{(j)}, x_2^{(j)}, \ldots, x_{l-1}^{(j)})$. The target t is the one hot encoding for x_l.

$$\begin{aligned} \text{loss}(x^{(j)}) &= \frac{1}{k^{(j)}-1} \sum_{l=2}^{k^{(j)}} L_{CE}(x_l^{(j)}) \\ \text{loss}(x^{(j)}) &= -\frac{1}{k^{(j)}-1} \sum_{l=2}^{k^{(j)}} \sum_{i=1}^{|T|} t_i \cdot \log p(x_{l,i}^{(j)}) \end{aligned} \tag{5}$$

Finally, the anomaly condition for Global heuristic is presented below:

$$\begin{aligned} &\text{loss}(x^{(j)}) > \text{UW} \\ &\text{loss}(x^{(j)}) > \text{Q3} + 1.5 \cdot \text{IQR} \end{aligned} \tag{6}$$

The effect of the Global heuristic is illustrated in Figure 3 in which we present a histogram of the loss values for the test sequences from the HDFS dataset, with the UW threshold colored in red. The yellow color represents the

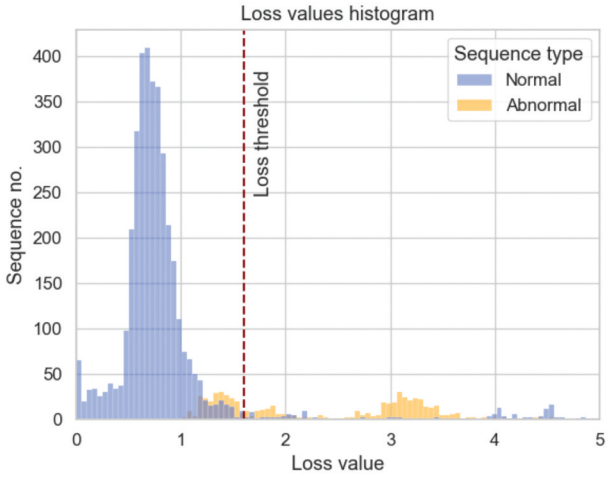


Figure 3. Histogram of testing sequences' loss values.

anomalies, and the blue represents the normal sequences. The horizontal axis constitutes the loss values, and the vertical axis is for the number of sequences in the histogram bucket.

Mixed heuristic

While the Extreme heuristic identifies sequences that have at least one low probability token and the Global heuristic identifies sequences that have a low probability overall, the Mixed heuristic combines these two to achieve a balance and optimize the F1 score.

A sequence is anomalous if it satisfies both heuristics, meaning it has at least one low probability token and the average loss is above the UW threshold obtained with IQR.

- While the first part of the heuristic extracts the potential anomaly candidates;
- The second one reorders them and predicts them as anomalous just the top abnormal ones based on the autoregression loss.

Mean Selection

In the experimental section, we empirically prove that the best results are achieved when the autoregressive language model training has not yet achieved convergence. The intuition behind this is that after the model has learned the easy normal sequences, it will also start learning harder and possibly anomalous sequences. Therefore, we employ early stopping to prevent overfitting.

To address this issue in an unsupervised manner, we employed Mean Selection, which aims to eliminate anomalies starting from the training phase. The main idea is represented by a dynamically adapted IQR method, combined with the Simulated Annealing (SA) method:

- The probability of an instance not being excluded from the training dataset in the current epoch is proportional to the distance between its loss and the batch loss.
- The probability is also proportional to the index of the current epoch as we want to eliminate fewer instances at the end of the training phase.

Mean Selection workflow

During the training phase, the following steps are applied for each mini-batch. One mini-batch represents a fixed number of instances for which the model weights are adapted accordingly.

We compute the loss for each sequence from the batch and subsequently the absolute difference between each loss and the average loss over the entire batch. The temperature of the SA method is lowered by a constant at the end of each epoch. These components influence the probability of each sequence being eliminated from the training dataset for the current epoch, in the following manner:

$$P(\mathrm{X}, \mathrm{UW}, \mathrm{T}) = e^{\frac{-|\mathrm{X}-\mathrm{UW}|}{1+\mathrm{T}}} \tag{7}$$

- X represents the vector of Cross-entropy losses for the sequences;
- UW is the upper whisker computed as $\mathrm{Q3} + 1.5 \cdot \mathrm{IQR}$;
- T is the temperature;
- the exponential function with a negative exponent is to ensure the probability has values between 0 and 1;

We generate random values between 0 and 1 for each sequence and eliminate the sequences whose elimination probability (Equation 7) is bigger than the generated value. As the temperature decreases, the elimination probability will also decrease leading to more sequences being included in the training. This approach has beneficial results because it reduces the False-Negative rate even when the autoregressive model achieves convergence, namely when the validation loss is reaching a plateau or is starting to increase.

Figure 4 displays in blue the percentage of the sequences eliminated during training, progressively for each epoch. The percentage of anomalies eliminated during training is colored in yellow. At the beginning, approximately 60% of anomalies are eliminated from training by removing just 20% from the total number of instances. Due to the Simulated Annealing, the percentage of

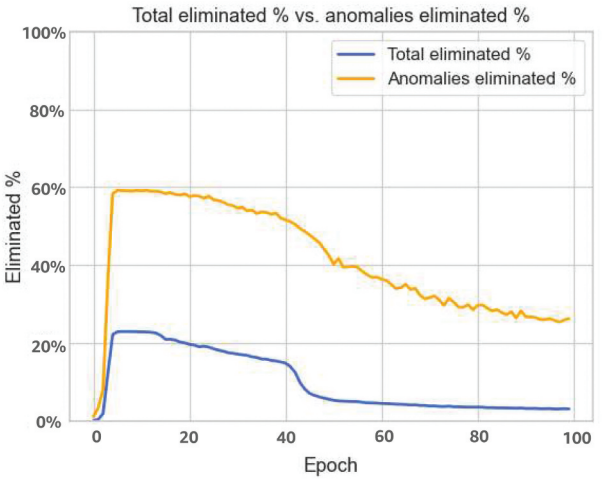


Figure 4. The percentage of eliminated sequences per epoch on the HDFS dataset.

sequences eliminated slowly decreases toward the end. Even so, there is still a high percentage of eliminated anomalies (>25%) which illustrates that Mean Selection is sensitive to outliers and improves the learning process by not letting the model see them too often.

Public Datasets

The loghub repository (Zhu et al. 2023) provides 17 real-world log datasets collected from different systems, such as supercomputers, operating systems, server applications, mobile systems, and finally, distributed systems. As far as we know, the loghub datasets are the main provider of data for system log anomaly detection and both industry and academia take advantage of their resources. We applied our experiments to three of the most cited datasets. They are split into two categories: distributed systems with HDFS (Xu et al. 2009), and supercomputers with BGL and Spirit, datasets collected by Sandia National Labs (SNL) (Oliner and Stearley 2007).

HDFS (Hadoop Distributed File System) is a dataset collected from a HDFS cluster formed by over 200 Amazon EC2 nodes. It consists of 11,175,629 message logs. The special thing about HDFS is that it is one of the few datasets that identifies their session by unique identifiers. Therefore, a session has varied size and corresponds to a *block_id*, reflecting a program execution in the HDFS system. The labeling is available per *block_id*. There are 575,062 total blocks of which 16,828 (2.93%) blocks are anomalies. The average length of a sequence is 18.4 logs.

There is a standard benchmark dataset extracted from HDFS with 28 log templates parsed from the logs. The split into training and testing subsets is fixed and can be used for fair comparison. However, the training dataset contains sessions only from normal blocks, and in real-life applications, there is no way to eliminate all anomalies from the training dataset without engaging in the labeling process.

BGL (BlueGene/L) is a dataset of logs collected from a supercomputer system at Lawrence Livermore National Labs (Oliner and Stearley 2007). It consists of 4,747,963 message logs. While the HDFS dataset has labels per session uniquely identified by *block_id*, BGL has labels for each message log. There are 348,460 (7.34%) message logs labeled as anomalous. The special thing about BGL is that it is the only dataset that has both ways for identifying sessions: by fixed window grouping applied on timestamps and by unique identifiers. By grouping by the unique identifiers, we end up with 69,252 sessions from which 31,375 (45.2%) are considered anomalies. Compared to HDFS, BGL has a higher percentage of anomalies, and it is more sensitive to fully unsupervised requirements, where we mixed both normal and abnormal sequences for training. The high anomaly percentage is caused by the fact that

a session is considered anomalous if at least a message log is labeled as an anomaly. The average length of a sequence is 68.1 logs.

Spirit is a dataset of logs collected from the Spirit supercomputer system at Sandia National Labs (Oliner and Stearley 2007). It consists of approximately 172 million anomalous message logs, but we use the 1 G version which contains a total of 7,983,344 message logs. There is no unique identifier for sessions as BGL has, so the grouping will be made only by a fixed window size on timestamps. Therefore, there will be different anomaly percentages when grouping by 1 h, 0.5 h, or grouping by 100 consecutive logs for example.

Preprocessing

All datasets have logs in a raw format, which we parse using the Drain (He et al. 2017) parser. We then apply a grouping phase that aims to gather parsed logs into sequences that form the dataset D used in the anomaly detection model to output the subset of anomalous sequences $A \subseteq D$. There are two approaches for grouping message logs, either based on a unique identifier or based on the timestamps.

Session window is a grouping strategy based on the identifier of logs. For HDFS, it is represented by *block_id* and for BGL by *node_id*. Because of the nature of grouping, the sequences have variable lengths depending only on the identifier. HDFS and BGL are the only datasets that have unique identifiers on logs. This is why they are the only ones for which we were able to compute the average session length in Table 1.

Fixed window grouping strategy needs the prerequisite to have the logs ordered according to their timestamps. The logs are grouped by fixed windows according to their occurrences. The *window_size* is a hyperparameter controlling how the dataset is built. It can express one of the two possibilities: a time span in the number of seconds for each window or the number of logs in a sequence.

Sliding window is similar to the Fixed window strategy, involving two hyperparameters: *window_size* and *step_size*. The *window_size* can be expressed in both seconds and number of logs, while the *step_size* represents the forwarding distance. This way, we can create an overlapping dataset for $step_size < window_size$ or extract a sample by setting up $step_size > window_size$.

Table 1. Session window datasets info.

Dataset	Logs No.	Vocab size	Sequences No.	Avg Sequence Length
HDFS	11,175,629	48	575,061	19.4
BGL	4,747,96	1,847	69,252	68.1

Training data selection strategies

Le and Zhang (2022) empirically proved that different selection strategies for splitting the dataset into training and validation data can affect the results. There are two types of training data selection strategies: random and chronological.

Random training data selection is a strategy that takes the build sequences created by one of the available grouping methods and consequently shuffles them. The training data is extracted from the shuffled dataset. The remaining data is used for testing purposes. The validation dataset used for the autoregressive language model needs to be extracted from the training dataset. There is no need for labeled data as the autoregressive training is fully unsupervised.

Chronological training data selection strategy does not shuffle the sequences. Instead, as a first step, the procedure orders the sequences chronologically and extracts the training data from the beginning. This way, future data cannot infiltrate the training dataset, and the experimental settings will be fair. In real-life applications, future activity is unknown, and a model should also be able to capture new types of anomalies.

For BGL, for example, the F1 score decreased from 92.7% to 42.6% using DeepLog and from 93.1% to 48.3% using LogAnomaly by changing from a random to a chronological selection strategy.

Experimental Settings

To achieve a fair comparison between the benchmark and our results, we use DeepLog (Du et al. 2017) for our experiments as it is one representative architecture used for semi-supervised system log anomaly detection. We build our solution on top of this architecture to achieve fully unsupervised training. Le and Zhang (2022) highlighted that even though other models achieve slightly better results, the problem is not solved by using a better architecture. Different ways of grouping, different splitting strategies, the anomaly ratio in the training data, and other data processing techniques are the most effective aspects that impact the results.

Hyperparameters

We select a default set of hyperparameters commonly used for these datasets. While hyperparameter tuning may indeed improve the results, we strongly stress that in production environments, this is not possible due to the lack of labeled data.

- *train_size* = 0.1 (for HDFS)/0.8 (for BGL); The fraction of sequences included in training. The other sequences will be part of the testing dataset and should not be touched during training;

- *valid_ratio* = 0.1; The percentage of training data used for validating the autoregressive task of next token prediction;
- *history_size* = 10 (for HDFS)/60 (for BGL); The number of tokens that form the context for the next token prediction task. The training input is formed by extracting *history_size* + 1 tokens with a rolling window. The model receives the first *history_size* tokens as input to output the *history_size* + 1 token. A sequence of k tokens results in $k - history_size$ instances;
- *n_warm_up_epoch* = 5; The number of epochs with low learning rate for warming up the model;
- *n_epochs_stop* = 10; The patience for early stopping;
- *batch_size* = 1024;
- *learning_rate* = 0.001(for HDFS)/0.0001(for BGL);
- Optimizer Adam (Kingma and Ba 2014) with parameters $\beta_1 = 0.9, \beta_2 = 0.98$.

Hardware resources

We have performed our experiments in a distributed cloud environment and deployed our experimental instance using SageMaker, an AWS machine learning service used by scientists to easily develop, train, and deploy models.

Our instance, p3.2 × large, is a GPU instance with 8 vCPUs, 61.0 GiB of memory, and an Intel Xeon E5–2686 v4 processor with a clock of 2.3 GHz. This instance has an Nvidia Tesla v100 GPU with 16 GiB vRAM. We took advantage of the integrated Jupyter notebook instance to conduct our experiments in a distributed environment provided with GPU computing power.

Results

Most previous work reported results on dataset splits in which the training data does not contain outliers, and are referred to as . In contrast to the previous approach, we include outliers in the training data, and these experiments are identified as *unlabeled data*. We display the results for each of the three proposed unsupervised heuristics. The *Best* keyword denotes the configurations in which the unsupervised heuristic was replaced with supervised fine-tuning on the testing dataset. These results represent an upper bound our heuristics can reach. For example, the *Best* row situated after an *Extreme* row refers to the results in which hyperparameter g is fine-tuned on the testing dataset (this is the SOTA result which we criticize and compare ourselves to). Besides the three unsupervised heuristics, fine-tuning both the g hyperparameter and the loss threshold in a semi-supervised method is a new contribution we did not find in the literature before.

We also introduce a new way of reporting results which is more robust to large volumes of duplicated logs. In literature, the F1 Score is calculated on all the sequences in the testing data. However, most of the sequences in the testing dataset are duplicates, and the large volume of normal sequences makes the SOTA results seem better than they are. Therefore, reporting the results on the unique testing sequences shows insight into the types of anomalies that are caught or not by our model.

In this section, we present the results on HDFS and BGL by using the session window grouping presented before. Next, we display our results on BGL using the fixed window grouping. Then, we compare our approach with SOTA solutions. Further results on the HDFS, BGL, and Spirit datasets are in Appendix A and Appendix B.

Session window datasets

HDFS

For each cell in Table 2 we display the results in the following format: , and LOSS TH is the *loss* threshold selected by the Global heuristic. We note that for the Extreme heuristic, our unsupervised result is the same as the semi-supervised one when training on *only normal data* because both methods find the same g hyperparameter. However, in the context of training with *unlabeled data*, both unsupervised and semi-supervised results collapse, leading to very low F1 scores. We criticize the SOTA methods in the literature, as they are highly dependent on using *only normal data* for training and this makes them unsuited for large-scale industrial use. Nevertheless, our proposed Mean Selection salvages the situation, proving that it can dynamically identify and eliminate a great part of outliers during training, making it harder for the model to learn the anomalous data. Therefore, our method improves the results in both scenarios, with the unsupervised results being close to the best semi-supervised results. We also manage to exceed the SOTA method based on the supervised selection of the g parameter (Du et al. 2017) using the Mixed heuristic.

Table 2. Comparison of semi-supervised methods with unsupervised heuristics on the HDFS; only normal/unlabeled data; non-unique and unique testing data.

	Non-unique testing data						unique testing data					
	w/o Mean Selection			w/ Mean Selection			w/o Mean Selection			w/ Mean Selection		
	G	LOSS TH	F1	G	LOSS TH	F1	G	LOSS TH	F1	G	LOSS TH	F1
Extreme	4/1	-	.91/.10	-/4	-	-/.90	4/1	-	.82/.36	-/4	-	-/.82
Best	4/4	-	.91/.22	-/6	-	-/.91	4/4	-	.82/.44	-/8	-	-/.**84**
Global	-	0.87/0.95	.92/.11	-	-/2.13	-/.75	-	0.87/0.95	.80/.26	-	-/2.13	-/.68
Best	-	1.04/0.59	.94/.31	-	-/1.26	-/.84	-	1.02/0.60	.82/.46	-	-/1.26	-/.74
Mixed	4/1	0.87/0.95	.94/.11	-/4	-/2.13	-/.74	4/1	0.87/0.95	.84/.26	-/4	-/2.13	-/.68
Best	3/1	0.97/0.59	**.96**/.31	-/7	-/1.01	-/.**92**	4/1	0.90/0.62	**.85**/.47	-/7	-/0.84	-/.**84**

Table 3. F1 scores for a statistical sample of five runs on the HDFS dataset.

	Non-unique testing data			Unique testing data		
	w/o Mean Selection		w/ Mean Selection	w/o Mean Selection		w/ Mean Selection
	only normal	unlabeled	unlabeled	only normal	unlabeled	unlabeled
Extreme	.88 (± .02)	.24 (± .02)	.88 (± .04)	.81 (± .01)	.47 (± .04)	.83 (± .00)
Best	.88 (± .02)	.24 (± .02)	.88 (± .04)	.81 (± .01)	.47 (± .04)	.83 (± .00)
Global	.91 (± .00)	.11 (± .00)	.71 (± .01)	.79 (± .00)	.28 (± .01)	.66 (± .01)
Best	.91 (± .00)	.31 (± .23)	.79 (± .03)	.80 (± .00)	.48 (± .02)	.74 (± .01)
Mixed	.91 (± .00)	.11 (± .00)	.71 (± .01)	.79 (± .00)	.28 (± .01)	.66 (± .01)
Best	**.93** (± .00)	**.32** (± .01)	**.88** (± .03)	**.82** (± .01)	**.49** (± .02)	**.85** (± .02)

We stress that the second part of Table 2 provides valuable information about the types of anomalies that are identified by the model. Generally, the results for the unique data are lower than the non-unique, showing that the *non-unique* results might not be reliable in terms of huge volumes of data. Notice that while the *Heuristic* results have the same parameters for both *non-unique* and *unique*, the *Best* results may use a different configuration.

To increase the confidence in our results, in Table 3 we provide metrics on a statistical sample of 5 runs for the HDFS dataset, using the same configuration but different random seeds for weight initialization and Mean Selection. The low variance of the results combined with the weight complexity of the LSTM model illustrates that our experiments are reliable.

In Figure 5, we display the F1 score evolution across epochs on the testing data. The dotted lines represent the results of the semi-supervised approach which has the best possible result as the *g* and *loss* thresholds are selected to optimize the F1 score directly on the test dataset. The full lines represent the results obtained by our unsupervised heuristics, bounded by the semi-

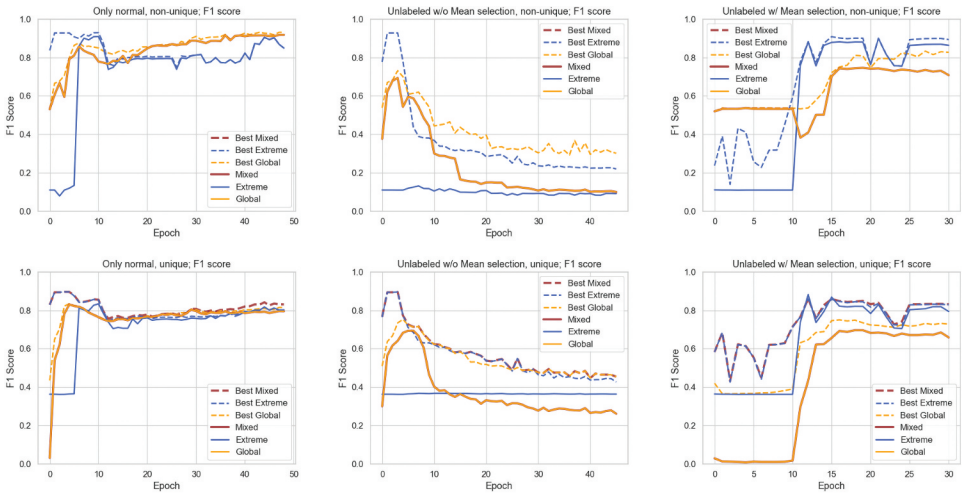


Figure 5. F1 scores for non-unique testing data (first row) and unique testing data (second row); HDFS dataset.

supervised ones. The overlapping of the semi-supervised and unsupervised lines is an indicator that the unsupervised heuristic achieves the best possible result. These experiments illustrate that the Extreme heuristic using the Elbow method is a powerful tool for selecting the *g* hyperparameter in an unsupervised manner, fairer and faster.

Figure 5 also illustrates that using unlabeled data for training makes the results collapse after the first few epochs, as the deep learning model starts to learn the anomalous data. In the third column, Mean Selection comes with two advantages: it salvages the results when anomalies are mixed in the training data, and it does not require using a very sensitive early stopping criterion to end training before the model starts overfitting.

We display the value of the hyperparameter *g* and *loss* threshold in Figure 6. It is important to note that even hyperparameter *g* selected using the semi-supervised fine-tuning depends on the number of epochs. The value of the best semi-supervised *g* is often greater than the unsupervised *g*. Furthermore, the values for the *g* and *loss* threshold normally decrease as the model learns more information, except when using Mean Selection, where there are some fluctuations in the beginning because this method tries to filter the outliers from the training data.

BGL

The significance of reporting metrics on the unique testing data is greater for the BGL dataset, as seen from Table 4. The F1 score on the non-unique testing data is greater than 0.99 when training on *only normal* data, which falsely suggests that the anomaly detection problem is solved. However, even in this privileged training scenario, the unique testing data F1 score is ~0.88, leaving

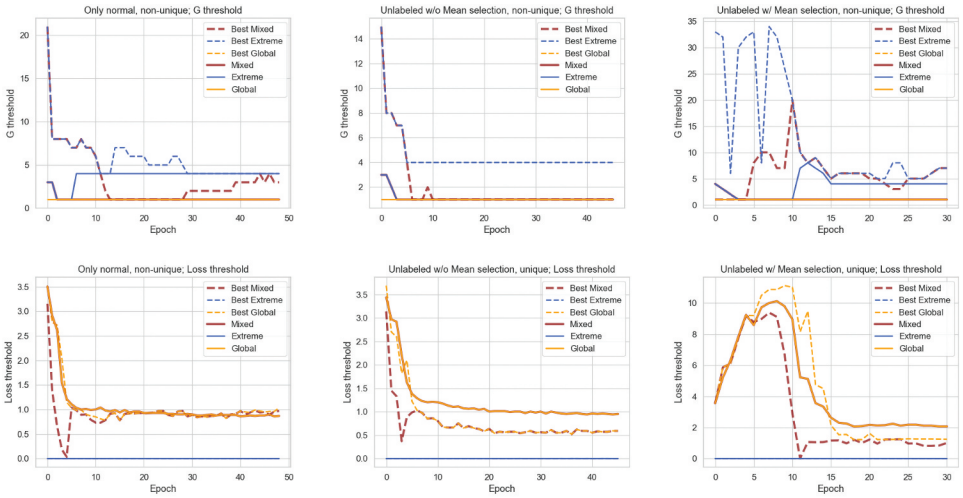


Figure 6. The evolution of the *g* hyperparameter and *loss* threshold; HDFS dataset.

Table 4. Comparison of semi-supervised methods with unsupervised heuristics on BGL; only normal/unlabeled data; non-unique and unique testing data.

	Non-unique testing data						Unique testing data					
	.w/o Mean Selection			w/ Mean Selection			.w/o Mean Selection			w/ Mean Selection		
	G	LOSS TH	F1	G	LOSS TH	F1	G	LOSS TH	F1	G	LOSS TH	F1
Extreme	1/1	-	**.99**/.03	-/1	-	-/.48	1/1	-	.87/.39	-/1	-	-/**.52**
Best	1/1	-	**.99**/.03	-/1	-	-/.48	1/1	-	.87/.41	-/1	-	-/**.52**
Global	-	4.47/1.87	**.99**/.01	-	-/0.54	-/.12	-	4.47/1.87	.87/.24	-	-/0.54	-/.15
Best	-	4.61/0.01	**.99**/.03	-	-/0.01	-/.48	-	4.62/0.01	**.88**/.41	-	-/0.01	-/**.52**
Mixed	1/1	4.47/1.87	**.99**/.01	-/1	-/0.54	-/.12	1/1	4.47/1.87	.87/.24	-/1	-/0.54	-/.15
Best	1/1	4.62/0.01	**.99**/.03	-/1	-/0.01	-/.48	1/1	4.62/0.05	**.88**/.39	-/1	-/0.01	-/**.52**

room for further improvements. Similar to the results on HDFS, mixing anomalies in the training data greatly affects the results, from an F1 score of 0.99 to 0.03. This stands as a powerful argument against SOTA methods in the literature that use *only normal* data for training, making them unusable for real large-scale unlabeled industrial datasets. Fortunately, Mean Selection increases the results, similar to the behavior on the HDFS dataset.

Table 4 also illustrates the chosen g hyperparameter on the BGL dataset. The value of 1 signifies the lowest possible tolerance for outliers, which shows that the normal sequences are very easy and are perfectly learned by the LSTM model. This is also the reason for the huge drop in F1 score when mixing anomalies in the training data. Furthermore, on the BGL dataset, both the Extreme heuristic and Global heuristic find the same g and a similar *loss* as the semi-supervised methods, which use the testing labels. This is another strong indicator that our proposed heuristics fit into the anomaly prediction pipeline.

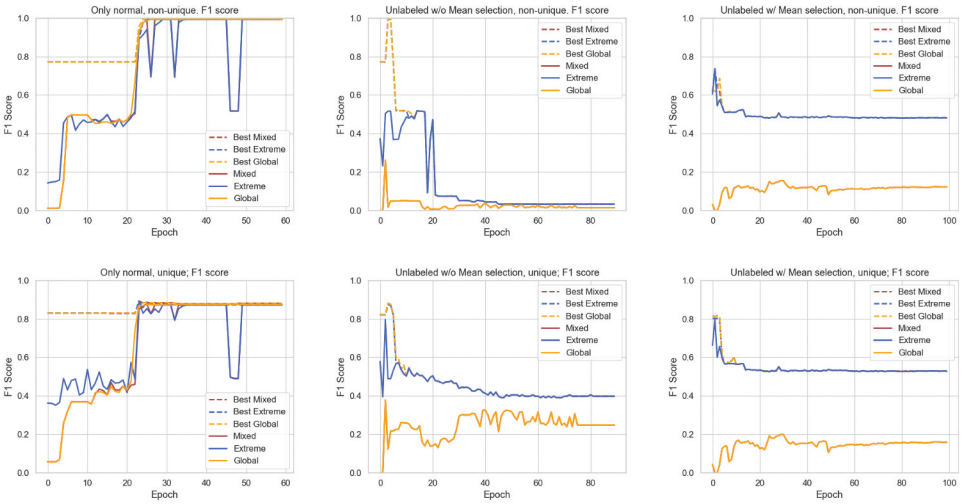


Figure 7. F1 scores for non-unique testing data (first row) and unique testing data (second row); BGL dataset, session grouping strategy.

Table 5. Comparison of semi-supervised methods with unsupervised heuristics on BGL; fixed *window_size* of 200 logs and 6 min; random/chronological selection.

	200 logs Grouping						6 minutes Grouping					
	w/o Mean Selection			w/ Mean Selection			w/o Mean Selection			w/ Mean Selection		
	G	LOSS TH	F1	G	LOSS TH	F1	G	LOSS TH	F1	G	LOSS TH	F1
Extreme	4/114	-	.30/.35	2/199	-	**.56**/.49	3/6	-	.30/.05	13/30	-	.43/.15
Best	231/177	-	.52/.45	8/199	-	**.56**/.50	2/199	-	**.56**/.11	6/41	-	.57/.16
Global	-	0.48/0.46	.06/.42	-	1.25/0.83	.06/.40	-	0.66/0.57	.03/.06	-	1.19/0.77	.80/.07
Best	-	0.01/0.98	.44/.48	-	0.01/1.35	.46/.48	-	0.01/10.51	.55/.21	-	16.82/5.81	.92/.17
Mixed	4/144	0.48/0.46	.05/.44	2/199	1.25/0.83	.06/.53	3/6	0.66/0.57	.03/.06	13/30	1.19/0.77	.40/.15
Best	242/191	0.01/0.35	**.56**/**.51**	5/189	0.01/1.13	**.56**/**.54**	2/7	0.01/5.26	**.56**/**.22**	13/111	16.82/4.22	**.92**/**.24**

In Figure 7 we see the running F1 Score on the BGL dataset. When training on only normal data, the results obtained by the unsupervised heuristic increase and match the semi-supervised ones after more than 20 epochs of training. In contrast to this, training with unlabeled data makes the semi-supervised results decrease from a reasonably high value to match the unsupervised results.

Further details about the experiments done on the BGL dataset using session window grouping are in A.2, including additional metrics such as precision and recall.

Fixed window

This grouping strategy is based on the timestamps of the logs. The logs are grouped by fixed windows according to their occurrences. The *window_size* is a hyperparameter of how the dataset is built. It can express one of two possibilities: a time span in the number of seconds for each window or the number of logs in a sequence.

We tested both training data selection strategies discussed in this thesis: random and chronological. The chronological is an assumption more frequently achieved in real-life applications as often we do not know future types of anomalies and can learn only on logs generated up to a certain point.

In Table 5 we compare the random and chronological selections under different configurations. One important thing to note is the variation of hyperparameter g including the g selected by the semi-supervised fine-tuning. This shows that there is no default value for hyperparameter g that works for any dataset, as we have multiple g thresholds chosen for different configurations of the same dataset.

Further details of the experiments done on the BGL dataset using fixed window grouping are in B.1. The experiments on the Spirit dataset are in B.2.

Comparison with the state of the art

In Table 6 we compare our method with State of the Art results in the literature. For a fair comparison, we use only normal data during training and report the results on the non-unique testing data, even though this creates an over-idealized

Table 6. Comparison with the state of the art on the HDFS and BGL datasets using session window; The values shown in bold denote the best results for each category; only normal data; non-unique testing data.

Type	Model	F1 HDFS	F1 BGL
Unsupervised	PCA	0.79	0.55
	Invariant Mining	**0.91**	0.91
Unsupervised heuristics (ours)	DeepLog, Extreme heuristic	0.91	0.99
	DeepLog, Global heuristic	0.92	0.99
	DeepLog, Mixed heuristic	**0.94**	0.99
Semi-Supervised	DeepLog, Best extreme	0.91	0.99
	LogFit	0.95	-
	LogAnomaly	**0.96**	0.99
Supervised	PLELog	0.93	0.99
	LogRobust	**0.98**	1.0

training scenario. Not having labeled data would render the supervised models useless and severely degrade the results of the semi-supervised models, as shown in previous sections.

Conclusion

The investigation in our research is motivated by the lack of system log anomaly detection solutions applicable to large-scale industrial datasets. The latest research trend for anomaly detection in the case of system logs deviates from the general unsupervised scenario. While it reports very good results, this trend imposes important difficulties when working with unlabeled, real-life datasets, leading us to address the question highlighted in the title of our work: "Are we on the right path?"

Our experiments illustrate that the benchmark on log system anomaly detection is valid only in certain circumstances, namely when labeled data is available. The experimental results in this paper show that the SOTA methods collapse when anomalous data is included in the training dataset. They also demonstrate that labeled data is necessary for the semi-supervised fine-tuning of hyperparameter g, as no default value is suitable for all datasets.

The supervised and semi-supervised methods are not usable in big tech companies, due to the huge costs associated with labeling. However, the unsupervised strategies in this work can replace the supervised phase of the semi-supervised methods in the literature. Experimental results show that unsupervised heuristics achieve almost optimal solutions while having the advantage of labeled data independence.

Moreover, this paper describes a training framework that is resilient to anomalies mixed in the training dataset, because industrial training datasets are not curated. Unlike the semi-supervised methods for which the results undergo a huge drop, our Mean Selection procedure makes the auto-regressive training robust to outliers.

Furthermore, this article establishes a benchmark more suitable for real-life applications, removing hidden assumptions propagated through the literature and reporting additional metrics for unique log sequences. This second aspect is important because such results reflect the diversity and sensitivity of model predictions independent of the volume of data, and the lower metric values underline that there is enough room for research in improving the system log anomaly detection methods.

Disclosure Statement

No potential conflict of interest was reported by the author(s).

ORCID

Ramona-Georgiana Albert http://orcid.org/0009-0005-2438-6213

Data Availability Statement

The data that support the findings of this study are openly available in the Loghub repository collected by Zhu et al. (2023), available at https://github.com/logpai/loghub, under an open license.

Highlights

- No suitable solution for large-scale system logs anomaly detection.
- Developing unsupervised heuristics for improving Deep Learning methods.
- Achieving results comparable with semi-supervised finetuning.
- Setting a more suitable benchmark and removing hidden assumptions found in literature.
- Proposing new metrics independent of the volume of system log anomalies.
- Establishing a testing framework applicable to large-scale industrial log systems.

References

Almodovar, C., F. Sabrina, S. Karimi, and S. Azad. 2024. Logfit: Log anomaly detection using fine-tuned language models. *IEEE Transactions on Network and Service Management* 21 (2):1715–23. doi: 10.1109/TNSM.2024.3358730.

Du, M., F. Li, G. Zheng, and V. Srikumar. 2017. Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In *Proceedings of the 2017 ACM Sigsac Conference on Computer and Communications Security*, 1285–98, New York, NY, USA: Association for Computing Machinery.

Farzad, A., and T. A. Gulliver. 2020. Unsupervised log message anomaly detection. *ICT Express* 6 (3):229–37. doi: 10.1016/j.icte.2020.06.003.

Guo, H., S. Yuan, and X. Wu. 2021. Logbert: Log anomaly detection via bert. In *2021 International Joint Conference on Neural Networks (IJCNN),* Shenzhen, China, 1–8.

He, P., J. Zhu, Z. Zheng, and M. R. Lyu. 2017. Drain: An online log parsing approach with fixed depth tree. In *2017 IEEE International Conference on Web Services (ICWS),* Honolulu, HI, USA, 33–40.

He, S., J. Zhu, P. He, and M. R. Lyu. 2016. Experience report: System log analysis for anomaly detection. In *2016 IEEE 27th International Symposium on Software Reliability Engineering (ISSRE),* Ottawa, ON, Canada, 207–18.

Hochreiter, S., and J. Schmidhuber. 1997. Long short-term memory. *Neural Computation* 9 (8):1735–80, November. doi: 10.1162/neco.1997.9.8.1735.

Kingma, D. P., and J. Ba. 2014. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.

Le, V.-H., and H. Zhang. 2022. Log-based anomaly detection with deep learning: How far are we? In *Proceedings of the 44th International Conference on Software Engineering*, 1356–67, New York, NY, USA: Association for Computing Machinery.

Liu, F. T., K. M. Ting, and Z.-H. Zhou. 2008. Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining,* Pisa, Italy, 413–22.

Lou, J.-G., Q. Fu, S. Yang, Y. Xu, and J. Li. 2010. Mining invariants from console logs for system problem detection. In *Proceedings of the 2010 Usenix Conference on Usenix Annual Technical Conference*, 24, USA: USENIX Association.

Ma, M. X., H. Y. Ngan, and W. Liu. 2016. Density-based outlier detection by local outlier factor on largescale traffic data. *Electronic Imaging* 28 (14):1–4. doi: 10.2352/ISSN.2470-1173.2016.14.IPMVA-385.

Meng, W., Y. Liu, Y. Zhu, S. Zhang, D. Pei, Y. Liu, and R. Zhou. 2019. Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs. In *Proceedings of the Twenty-eighth International Joint Conference on Artificial Intelligence, IJCAI-19,* Macao, China, 4739–45. International Joint Conferences on Artificial Intelligence Organization.

Mikolov, T., K. Chen, G. Corrado, and J. Dean. 2013. Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.

Muniyandi, A., R. Ramachandran, and R. Rajaram. 2012. Network anomaly detection by cascading k-means clustering and c4.5 decision tree algorithm. *Procedia Engineering* 30:174–82, 12. doi: 10.1016/j.proeng.2012.01.849.

Nedelkoski, S., J. Bogatinovski, A. Acker, J. Cardoso, and O. Kao. 2020. Self-attentive classification-based anomaly detection in unstructured logs. In *2020 IEEE International Conference on Data Mining (ICDM),* Sorrento, Italy, 1196–201.

Nguyen, K. A., S. Schulte Im Walde, and N. T. Vu. 2016. Integrating distributional lexical contrast into word embeddings for antonym-synonym distinction. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (volume 2: Short papers)*, ed. K. Erk and N. A. Smith, 454–59, Berlin, Germany: Association for Computational Linguistics, August.

Oliner, A., and J. Stearley. 2007. What supercomputers say: A study of five system logs. In *37th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN'07),* Edinburgh, UK, 575–84.

Samariya, D., and A. Thakkar. 2023. A comprehensive survey of anomaly detection algorithms. *Annals of Data Science* 10 (3):829–50, Jun 1.

Xu, W., L. Huang, A. Fox, D. Patterson, and M. I. Jordan. 2009. Detecting large-scale system problems by mining console logs. In *Proceedings of the ACM SIGOPS 22nd Symposium on Operating Systems Principles*, 117–32, New York, NY, USA: Association for Computing Machinery.

Yamanaka, Y., T. Takahashi, T. Minami, and Y. Nakajima. 2024. Logelectra: Self-supervised anomaly detection for unstructured logs. *arXiv preprint arXiv:2402.10397*.

Yang, L., J. Chen, Z. Wang, W. Wang, J. Jiang, X. Dong, and W. Zhang. 2021. Plelog: Semi-supervised log-based anomaly detection via probabilistic label estimation. *2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion),* Madrid, Spain, 230–31.

Yu, X., P. Joshi, J. Xu, G. Jin, H. Zhang, and G. Jiang. 2016. Cloudseer: Workflow monitoring of cloud infrastructures via interleaved logs. *ACM SIGPLAN Notices* 51 (4):489–502, 3. doi: 10.1145/2954679.2872407.

Zhang, C., T. Jia, G. Shen, P. Zhu, and Y. Li. 2024. Metalog: Generalizable cross-system anomaly detection from logs with meta-learning. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, New York, NY, USA: Association for Computing Machinery.

Zhang, X., Z. Li, J. Chen, X. He, R. Yao, J.-G. Lou, and Q. Cheng. 2019. Robust log-based anomaly detection on unstable log data. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering,* New York, NY, USA, 807–17.

Zhu, J., S. He, P. He, J. Liu, and M. R. Lyu. 2023. Loghub: A large collection of system log datasets for ai-driven log analytics. *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, Florence, Italy, 355–66.

Appendices

Appendix A. Session window datasets

A.1. *HDFS*

In Figure A1, we illustrate the evolution of the g threshold chosen by the **Extreme** heuristic. The usual evolution begins with high values for g in the first epochs due to model uncertainty. As the model learns to predict the next token better, the value of hyperparameter g decreases until it reaches a stable point at the end of the training.

Figure A2 shows the evolution of the *loss* threshold chosen by the **Global** heuristic. Because in the first few epochs, the distribution of the loss on the test sequences is too noisy, we start with consecutive loss histograms starting from epoch 17 on the first row and from epoch 40 on the second row. The *loss* threshold chosen by our unsupervised heuristic separates a portion of anomalies, while also maintaining high precision.

In Figure A3 we illustrate the evolution of the Autoregressive Language Model next token probability distribution. The first row represents the evolution when Mean Selection is disabled, while the second row displays the evolution when we are using the **Mean Selection** strategy. During the final epochs, the distribution is leptokurtic and left skew meaning that most of the tokens are predicted right by the Autoregressive Language Model with a high probability. During the first epochs, Mean Selection ignores the high loss instances and predicts with a low probability the actual next tokens. In the end, the distribution in both cases has similar shapes, with the difference that Mean Selection has a higher peak on the left side, meaning that there are more anomalies skipped during the learning process.

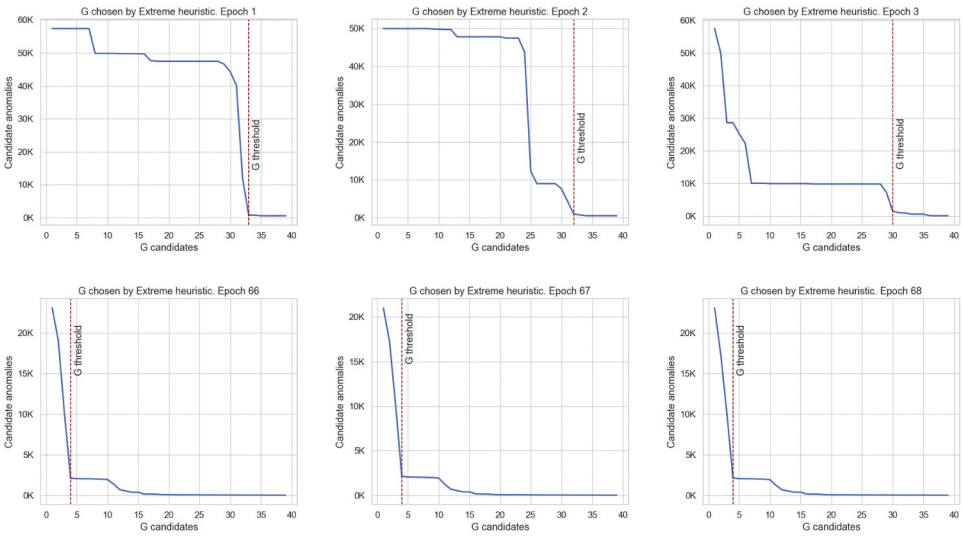


Figure A1. The evolution of the G threshold. HDFS dataset.

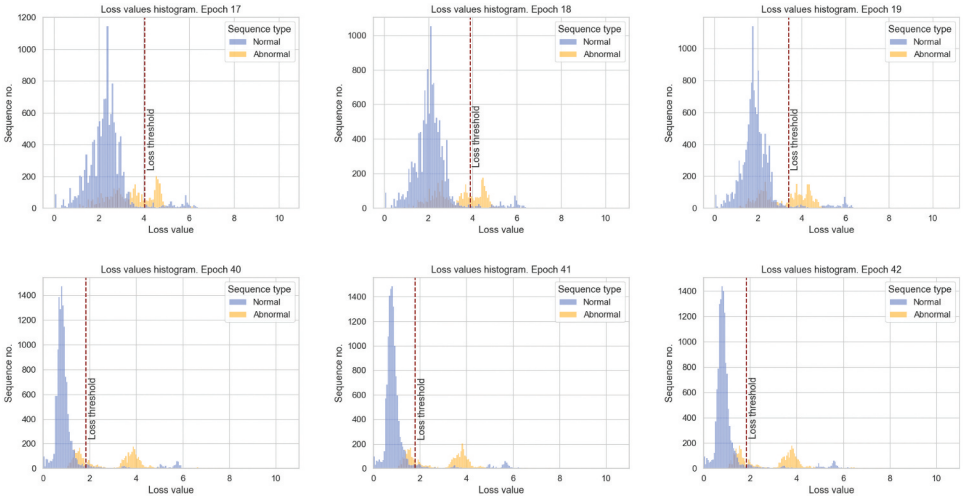


Figure A2. The evolution of the loss threshold. HDFS dataset.

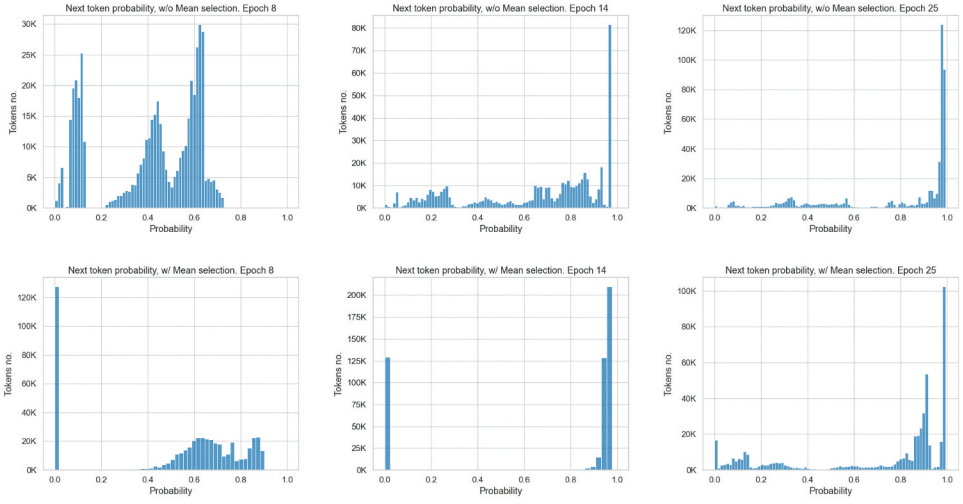


Figure A3. The evolution of the autoregressive language model next token probability. HDFS dataset.

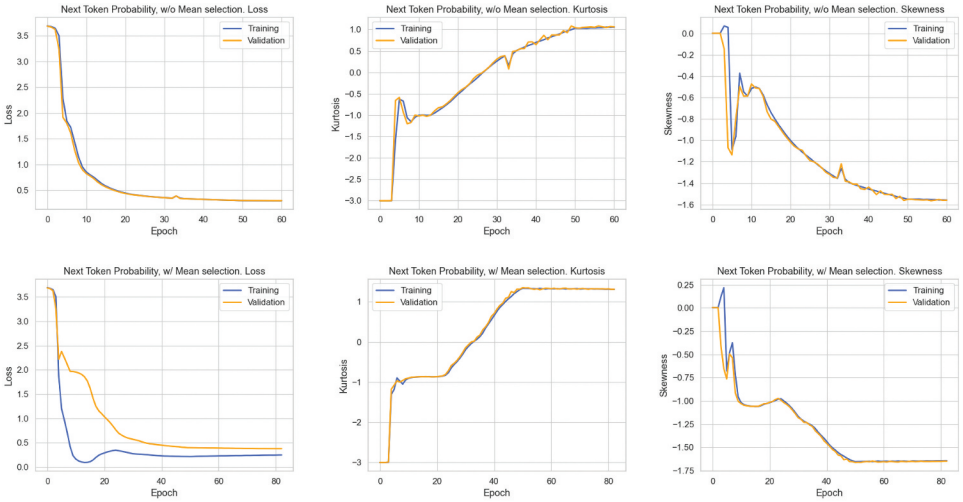


Figure A4. The evolution of the autoregressive language model next token probability distribution. HDFS dataset.

In Figure A4 we monitored the loss and the distribution metrics, kurtosis, and skewness for training and validation datasets.

A.2. *BGL*

In Table A1 we compare training with *only normal* training data versus training with unlabeled data and display the metrics on the non-unique testing sequences. In addition to the previous results in the main paper, we also include precision and recall. Therefore, here it can be clearly seen that the reason for the huge drop in F1 score from 0.99 to 0.03 is due to a collapse in recall.

Table A1. Comparison of semi-supervised methods with unsupervised heuristics on BGL; only normal/unlabeled data; non-unique testing data.

	w/o Mean Selection					w/ Mean Selection				
	G	TH_LOSS	F1	P	R	G	TH_LOSS	F1	P	R
Extreme	1/1	-	**.99**/.03	**.99**/**.83**	**.98**/.01	-/1	-	-/**.48**	-/.60	-/**.39**
Best	1/1	-	**.99**/.03	**.99**/**.83**	**.98**/.01	-/1	-	-/**.48**	-/.60	-/**.39**
Global	-	4.47/1.87	**.99**/.01	**.99**/.71	**.98**/.007	-	-/0.54	-/.12	-/**.70**	-/.06
Best	-	4.61/0.01	**.99**/.03	**.99**/**.83**	**.98**/.01	-	-/0.01	-/**.48**	-/.60	-/**.39**
Mixed	1/1	4.47/1.87	**.99**/.01	**.99**/.71	**.98**/.007	-/1	-/0.54	-/.12	-/**.70**	-/.06
Best	1/1	4.62/0.01	**.99**/.03	**.99**/**.83**	**.98**/.01	-/1	-/0.01	-/**.48**	-/.60	-/**.39**

Table A2. Comparison of semi-supervised methods with unsupervised heuristics on BGL; only normal/unlabeled data; unique testing data.

	w/o Mean Selection					w/ Mean Selection				
	G	TH_LOSS	F1	P	R	G	TH_LOSS	F1	P	R
Extreme	1/1	-	.87/.39	.92/.78	.82/.26	-/1	-	-/**.52**	-/.59	-/**.47**
Best	1/1	-	.87/**.41**	.92/**.79**	.82/**.27**	-/1	-	-/**.52**	-/.59	-/**.47**
Global	-	4.47/1.87	.87/.24	.93/.69	**.83**/.15	-	-/0.54	-/.15	-/**.71**	-/.08
Best	-	4.62/0.01	**.88**/**.41**	**.94**/**.79**	.82/**.27**	-	-/0.01	-/**.52**	-/.60	-/.46
Mixed	1/1	4.47/1.87	.87/.24	.93/.69	.82/.15	-/1	-/0.54	-/.15	-/**.71**	-/.08
Best	1/1	4.62/0.005	**.88**/.39	**.94**/.78	.82/.26	-/1	-/0.01	-/**.52**	-/.60	-/.46

Table A2 presents the results of the unique testing data for the same configuration as Table A1. It is important to note here that the recall for unlabeled data on unique testing data, is much higher than the recall on the non-unique testing data, 0.26 vs. 0.01. This shows that there is a big volume of identical outliers that are not identified by our model as they are easily learned when mixed in the training dataset.

Appendix B. Fixed window datasets

B.1. *BGL*

In Table B1, we compare the results on the BGL dataset for a chronological selection of testing sequences compared with a random selection. The comparison is done in the more correct scenario, so we use unlabeled data for training instead of *only normal*. When using a chronological selection, the log sequences from the testing dataset are chronologically after the log sequences from the training dataset. This is not the case when using a random selection,

Table B1. Comparison of semi-supervised methods with unsupervised heuristics on BGL; fixed *window_size* of 200 logs; unlabeled data; random/chronological selection.

	w/o Mean Selection					w/ Mean Selection				
	G	TH_LOSS	F1	P	R	G	TH_LOSS	F1	P	R
Extreme	4/114	-	.30/.35	.23/.24	.47/.**72**	2/199	-	.**56**/.49	.39/.48	.**98**/.49
Best	231/177	-	.52/.45	.48/.34	.**57**/.69	8/199	-	.**56**/.50	.39/.47	.**98**/.51
Global	-	0.48/0.46	.06/.42	.32/.32	.03/.59	-	1.25/0.83	.06/.40	.14/.32	.04/.**54**
Best	-	0.01/0.98	.44/.48	.29/.51	1/.46	-	0.01/1.35	.46/.48	.29/.56	1/.51
Mixed	4/114	0.48/0.46	.05/.44	.40/.36	.04/.57	2/199	1.25/0.83	.06/.53	.14/.60	.04/.47
Best	242/191	0.01/0.35	.**56**/.**51**	.**55**/.**54**	.**57**/.48	5/189	0.01/1.13	.**56**/.**54**	.**40**/.**59**	.97/.48

Table B2. Comparison of semi-supervised methods with unsupervised heuristics on BGL dataset; fixed *window_size* of 200 logs; chronological selection; only normal/unlabeled data.

	w/o Mean Selection					w/ Mean Selection				
	G	TH_LOSS	F1	P	R	G	TH_LOSS	F1	P	R
Extreme	5/114	-	.56/.35	.38/.24	.**98**/.**72**	-/199	-	-/.49	-/.48	-/.49
Best	8/177	-	.57/.45	.39/.34	.**98**/.69	-/199	-	-/.50	-/.47	-/.51
Global	-	0.46/0.46	.76/.42	.**89**/.32	.65/.59	-	-/0.83	-/.40	-/.32	-/.**54**
Best	-	0.38/0.98	.76/.48	.87/.51	.68/.46	-	-/1.35	-/.48	-/.56	-/.51
Mixed	5/114	0.46/0.46	.76/.44	.**89**/.36	.65/.57	-/199	-/0.83	-/.53	-/.**60**	-/.47
Best	1/191	0.39/0.35	.**77**/.**51**	.86/.**54**	.68/.48	-/5	-/1.13	-/.**54**	-/.59	-/.48

Table B3. Comparison of semi-supervised methods with unsupervised heuristics on BGL; fixed *window_size* of 6 min logs; unlabeled data; random/chronological selection.

	w/o Mean Selection					w/ Mean Selection				
	G	TH_LOSS	F1	P	R	G	TH_LOSS	F1	P	R
Extreme	3/6	-	.30/.05	.23/.02	.44/.**96**	13/30	-	.43/.15	.32/.09	.62/.**82**
Best	2/199	-	.**56**/.11	.**39**/.10	.**99**/.14	6/41	-	.57/.16	.40/.10	.**99**/.**82**
Global	-	0.66/0.57	.03/.06	.06/.03	.02/.46	-	1.19/0.77	.80/.07	.74/.04	.88/.57
Best	-	0.01/10.51	.55/.21	.38/.33	1/.16	-	16.82/5.81	.92/.17	.**99**/.11	.86/.46
Mixed	3/6	0.66/0.57	.03/.06	.06/.03	.02/.42	13/30	1.19/0.77	.40/.15	.51/.09	.32/.40
Best	2/7	0.01/5.26	.**56**/.**22**	.**39**/.**18**	.**99**/.28	13/111	16.82/4.22	.**92**/.**24**	.**99**/.**19**	.86/.34

Table B4. Comparison of semi-supervised methods with unsupervised heuristics on BGL; fixed *window_size* of 6 min logs; chronological selection; only normal/unlabeled data.

	w/o Mean Selection					w/ Mean Selection				
	G	TH_LOSS	F1	P	R	G	TH_LOSS	F1	P	R
Extreme	39/6	-	.33/.05	.21/.02	**.90**/**.96**	-/30	-	-/.15	-/.09	-/**.82**
Best	189/199	-	.38/.11	.29/.10	.56/.14	-/41	-	-/.16	-/.10	-/**.82**
Global	-	0.61/0.57	.10/.06	.06/.03	.94/.46	-	-/0.77	-/.07	-/.04	-/.57
Best	-	8.64/10.51	.75/.21	.73/**.33**	.78/.16	-	-/5.81	-/.17	-/.11	-/.46
Mixed	39/6	0.61/0.57	.37/.06	.23/.03	**.90**/.42	-/30	-/0.77	-/.15	-/.09	-/.40
Best	40/7	8.64/5.26	**.77**/**.22**	**.79**/.18	.76/.28	-/111	-/4.22	-/**.24**	-/**.19**	-/.34

Table B5. Comparison of semi-supervised methods with unsupervised heuristics on Spirit; fixed *window_size* of 200 logs; unlabeled data; random/chronological selection.

	w/o Mean Selection					w/ Mean Selection				
	G	TH_LOSS	F1	P	R	G	TH_LOSS	F1	P	R
Extreme	52/587	-	.75/.68	.77/.81	.73/.58	16/13	-	.78/**.93**	.77/.87	.78/**1**
Best	16/567	-	.89/.92	.81/.85	**1**/.98	1/13	-	.89/**.93**	.81/.87	**1**/**1**
Global	-	1.43/1.33	.37/.56	.87/.78	.24/.44	-	4.14/1.90	.35/.57	.79/.79	.24/.44
Best	-	0.07/0.14	.94/.93	.91/.88	.97/**1**	-	0.25/0.18	**.94**/**.93**	**.93**/**.88**	.95/**1**
Mixed	52/587	1.43/1.33	.37/.56	.87/.78	.24/.43	16/13	4.14/1.90	.35/.57	.79/.79	.23/.44
Best	15/439	0.07/0.15	**.95**/**.95**	**.93**/**.89**	.96/1	1/1	0.25/0.18	**.94**/**.93**	**.93**/**.88**	.95/**1**

which leads to future data leaking into the training dataset, therefore artificially improving the results.

For the experiments in Table B2, we use the more correct setting, which is a chronological selection of the training data, and we compare training on *only normal* or unlabeled data.

Similarly, for the experiments in Table B3 we use unlabeled data as the more correct setting and compare random versus chronological selection within a fixed window of 6 min worth of

Table B6. Comparison of semi-supervised methods with unsupervised heuristics on Spirit; fixed *window_size* of 200 logs; chronological selection; only normal/unlabeled data.

	w/o Mean Selection					w/ Mean Selection				
	G	TH_LOSS	F1	P	R	G	TH_LOSS	F1	P	R
Extreme	429/587	-	.73/.68	.83/.81	.65/.58	-/13	-	-/**.93**	-/.87	-/**1**
Best	399/567	-	**.93**/.92	.88/.85	**1**/.98	-/13	-	-/**.93**	-/.87	-/**1**
Global	-	8.46/1.33	.91/.56	**.90**/.78	.73/.44	-	-/1.90	-/.57	-/.79	-/.44
Best	-	0.49/0.14	**.93**/.93	.88/.88	**1**/**1**	-	-/0.18	-/**.93**	-/**.88**	-/**1**
Mixed	429/587	8.46/1.33	.69/.56	.87/.78	.57/.43	-/13	-/1.90	-/.57	-/.79	-/.44
Best	1/439	0.52/0.15	**.93**/**.95**	.88/**.89**	**1**/**1**	-/1	-/0.18	-/**.93**	-/**.88**	-/**1**

Table B7. Comparison of semi-supervised methods with unsupervised heuristics on Spirit; fixed *window_size* of 6 min logs; unlabeled data; random/chronological selection.

	w/o Mean Selection					w/ Mean Selection				
	G	TH_LOSS	F1	P	R	G	TH_LOSS	F1	P	R
Extreme	4/2	-	**.80**/.33	.76/.31	.84/.35	4/2	-	.80/.51	.76/.43	.85/63
Best	14/1	-	**.80**/**.71**	.79/**.55**	.81/**.99**	336/1	-	**.82**/**.71**	**.92**/**.55**	.73/**.99**
Global	-	4.59/4.13	.05/.10	.40/.38	.02/.06	-	7.31/8.81	.04/.14	.40/.36	.02/.09
Best	-	2.13/0.01	.72/.70	.56/.54	**.99**/**.99**	-	0.01/0.01	.73/.71	.57/**.55**	**.99**/**.99**
Mixed	4/2	4.59/4.13	.04/.11	.40/.28	.02/.06	4/2	7.31/8.81	.03/.14	.36/.37	.02/.08
Best	45/1	0.01/0.01	.79/.70	**.82**/.54	.77/**.99**	128/1	0.01/0.01	.80/.71	.68/**.55**	.97/**.99**

Table B8. Comparison of semi-supervised methods with unsupervised heuristics on Spirit; fixed *window_size* of 6 min logs; chronological selection; only normal/unlabeled data.

	w/o Mean Selection					w/ Mean Selection				
	G	TH_LOSS	F1	P	R	G	TH_LOSS	F1	P	R
Extreme	2/1	-	.83/.33	.74/.31	**.96**/.35	-/2	-	-/.48	-/.42	-/.58
Best	4/1	-	.85/.**71**	.78/.**55**	.92/.**99**	-/1	-	-/.**71**	-/.**55**	-/.**99**
Global	-	4.76/4.13	.83/.10	**.97**/.38	.73/.06	-	-/12.09	-/.13	-/.29	-/.07
Best	-	1.16/0.01	.86/.70	.84/.54	.87/.**99**	-	-/0.01	-/.**71**	-/.**55**	-/.**99**
Mixed	2/1	4.76/4.13	.83/.11	**.97**/.28	.73/.06	-/2	-/.71	-/.13	-/.29	-/.07
Best	3/1	0.71/0.01	**.87**/.70	.87/.54	.87/.**99**	-/1	-/0.01	-/.70	-/.54	-/.**99**

logs. In this case, the length of the fixed window is variable and depends on the number of consecutive logs within the time interval. In Table B4 we compare training on *only normal* and unlabeled data.

B.2. *Spirit*

In Table B5 and Table B6, we present the results on the Spirit dataset, with a fixed window grouping with 200 logs. We compare the random order with chronological selection in the context of unlabeled data, and *only normal* and unlabeled data in the context of chronological selection.

Similarly, in Table B7 and Table B8, we present the results when using a fixed window grouping of 6 min of logs, under similar conditions as above.